

A Survey of Current Object-Oriented Databases

Mansour Zand

Val Collins

Dale Caviness

Computer Science Department
University of Nebraska at Omaha

Abstract

Object-oriented concepts form a good basis for the data models required for next-generation database applications such as CAD/CAE/CASE/CAM systems, knowledge-based systems, multimedia, etc. Many object-oriented databases are available commercially or are being developed by industry or academic research facilities. This paper attempts to compare some of these products using fourteen criteria. The selected criteria are major factors required for the successful design of an object-oriented database system. They are grouped into five different categories. The result of the survey is summarized in a table.

Key words: object-oriented database, OODBMS survey, object-oriented terminology.

Introduction

An object-oriented database management system (OODBMS) is a database system based on object-oriented (O-O) data model concepts. In this approach, a database is considered as a collection of objects whose behavior, state, and relationships are stored as a physical entity (Kim, 1991). In contrast, the classical record-oriented data models (relational, network, or hierarchical), view data as a collection of record types or relations, each having a collection of records or tuples stored in a file. One of the goals of object-oriented modeling is to maintain a direct correspondence between real-world and database objects that do not lose their integrity and identity, and can easily be identified and operated upon. O-O concepts form a good basis for a data model required for the next-generation database applications such as CAD/CAE/CASE/CAM systems, knowledge-based systems, multimedia information systems, and advanced human-interface systems (Vorwerk, 1994).

Currently, there are many object-oriented database systems being developed by commercial vendors, industrial research laboratories, and universities around the world. Presently, there is not a single standard for O-O concepts, so the object-oriented approach is still being refined. Because of this lack of a formal standard, there is some skepticism with the feasibility of building commercial object-oriented database systems.

However, many believe that with further research and development, a standard object-oriented data model will emerge, an object-oriented database language to support the data model will follow, and ultimately an object-oriented database system will be constructed to support the language. Not surprisingly, this is essentially the approach taken in all current object-oriented database systems (Atwood, 1994).

This paper is organized in four sections. In the first section, a brief discussion on the advantages of OODBMS technology is provided. The second section provides a list of survey criteria, a short description, and the significance of each one. This section also contains definitions of some of the terminology used in the paper. Sections three and four present results of a survey in descriptive and tabular forms, respectively. This survey by no mean is a comprehensive study of the selected OODBMSs. The first goal of the survey is informational in nature. Its objective is to determine if each system has provided the essential and significant features required by an OODBMS. The second goal of the paper is to provide some details and technical information of how a specific feature is supported by each system.

Advantages of OODBMSs

Traditional relational database management systems (RDBMS) have been developed to meet the needs of traditional business applications. The data needed by these applications, along with the operations to be performed on those data, fit nicely into the package offered by relational database systems. With the introduction of next-generation database applications such as those described above, the requirements for data storage and manipulation are not well supported for these applications. There is a need for the ability to deal with complex data such as images, voice, and textual documents. Design classes and objects in an OODBMS represent real-world design entities and give a much better feel for the mechanics of the problem than do a set of flat tables as in a RDBMS (Ahmed et al., 1992).

One major advantage of the O-O concept is that objects can be reused in different programs. Programs can be built in a fraction of the time it takes to build them from scratch. Software ob-

jects can be built to represent a wide range of entities, from an abstract concept to a specific object.

In general, OODBMSs provide the following advantages over RDBMSs:

1. A more realistic and powerful data model.
2. Handles complex objects.
3. Inheritance relationships among sets of entities.
4. Associates object behaviors with object definitions at the schema level.
5. Implicit joins along aggregation hierarchies, based on object identity.
6. Version mechanisms.
7. Better transaction management and concurrency.
8. More expressive query language.
9. Better support for cooperative work.

Survey Factors

In this survey, 15 factors have been considered. These factors, presented in two tables, have been grouped into five categories:

1. Principal O-O features including primary uses, version management, recovery, transaction management, composite objects, multiple inheritance, and concurrency control;
2. Advanced O-O features including distributed support, dynamic evolution, and multimedia data management;
3. System Interface features including language interfaces and relational/object-oriented basis;
4. Status features including current platforms and operational status, and
5. Special features not covered above.

Following is a brief discussion on the significance of some of the selected criteria. The survey table indicates whether a given feature is supported, and in some cases more information is provided.

Version Management An OODBMS should provide primitives to manage multiple versions and provide persistence data/objects. Version control and management is required to access multiple versions of an object and also to access and maintain historical database. The summary table indicates whether version management is supported or not.

Recovery A recoverable DBMS is one that can get to a consistent state after a failure. Recovery in OODBMS is more complex due to the presence of persistent objects, long duration transactions, and a variety of data/objects. The survey table shows how recovery is implemented.

Transaction Management This feature deals with the problems of keeping the database in a consistent state when concurrent access and failures occur. An OODBMS must provide transactions on an equivalent mechanism for concurrency control and recovery not only for regular transactions but also for long duration ones. The major issues in transaction management of long-duration transactions are atomicity condition criteria. The survey table indicates availability of transaction management for long-duration transactions.

Composite Objects One of the major promises of OODBMSs is to provide the ability to aggregate composite objects for the purpose of storage and operation. Composite objects are made up of several objects connected by hierarchical structure. Most of the surveyed OODBMSs in this study fulfill this promise. The survey table shows those which can deal with more complex objects and those which have some sort of constraints.

Multiple Inheritance Inheritance is the ability to inherit the objects, methods, and constraints from one or more classes of object. Inheritance promotes reuse, abstraction, and facilitates maintenance. It is a fundamental feature that distinguishes object-oriented from other programming languages. Multiple inheritance allows all features of several super classes to be inherited by one class. The table indicates whether multiple inheritance is supported or not.

Concurrency Control Protocols The concurrency control provides a mechanism to protect database integrity while allowing the interleaved execution of multiple concurrent transactions. Concurrency control in OODBMSs is more complex than in traditional DBMSs. The main factors for this added complexity are the existence of long-duration transaction and the coexistence of several valid states (versions) of an object. The majority of the surveyed OODBMSs provides concurrency control using locking protocols. The survey table indicates the

type of protocol used in concurrence control and provides some detail on protocol implementation.

Distributed Support Applications that require object-oriented database capabilities are generally highly distributed. Furthermore, distribution of objects increases availability and performance. Distribution of resources poses the problems of naming, protection, and consistency. Our survey indicates that less than half of the OODBMSs in this study support distribution of objects.

Dynamic Schema Evolution A schema is a definition of the structure and behavior of an object. A schema could evolve during its existence in order to capture the changing environment. Redefinition of object classes during run time is called dynamic evolution of database (object) schema. Dynamic evolution is a powerful feature in the design and modification of database schema. In some of the OODBMSs, dynamic modifications on the contents (i.e., attribute, methods) and structure (i.e., classes and their relationships) are possible. The table indicates whether this powerful feature is supported or not.

Multimedia Data Management Multimedia data deal with a large number of images, documents, and audio and video information. Multimedia objects are in general very large and might be presented based on a time reference (temporal data) or be continuous. Non-structured multimedia data can be represented as objects in an OODBMS.

Extensibility, flexibility, and efficiency are the three major objectives in an OODBMS that supports the capture, storage, and presentation of multimedia data (Kim, 1989). The first two objectives are also among the primary design objectives of OODBMSs. The presence of efficiency as one of the primary design objectives is crucial for supporting multimedia data.

Efficiency is important in transferring large amounts of data and storage utilization. Therefore, an OODBMS that supports multimedia data should provide a more efficient data storage and retrieval system.

Relational or OO-Based Design Some of the surveyed systems are not truly designed on the

OO-based model. They are built on the top of a relational database system, and objects are mapped into or from one or more tuples in relations. Some of the OODBMSs are actually an extension of relational database system and use messages and objects for language design. The survey table indicates whether the underlying repository model is relational or OO-based.

Since the concepts and terminology of O-O methodology are not universal, the next section attempts to provide a consistent set of definitions for understanding O-O programming and databases.

Definition and Terminologies

1. **Data abstraction and encapsulation:** The ability to define a set of operators (methods) that can be applied to objects of a particular object class (or object type). Objects are manipulated by operations that are defined for the object type, and the internals are hidden from external access.
2. **Object Identity:** The OODBMS defines and maintains unique identifiers for objects. This allows objects which have the same attributes and values to exist without having to define unique keys for entity instances.
3. **Inheritance:** A subclass inherits both data attributes and methods from its superclasses. This allows class structures to be reused and/or redefined in terms of existing ones.
4. **Message Passing:** A message consists of one or more objects followed by a method to be applied to those objects. Objects interact by invoking each other's methods.
5. **Operator Overloading:** The ability to use the same operation name on different object types. For example, the same print operation can be applied to both a PC screen and to a printer.
6. **Class:** An object that corresponds to the traditional notion of a data type. A class acts as a template for defining a set of instance attributes, a set of messages for external use, and a set of methods that apply to objects of the class.
7. **Method:** An operation or function that can be performed on objects of a particular class. Methods define the "public interface" for objects of that class.
8. **Object:** Corresponds to the notion of a variable in the programming sense; therefore, it has a type (class) and a value (state). There are six common types of objects:
 - a) **Simple object:** Contains no multi-valued properties and no object properties. It is represented by a single relation.
 - b) **Compound object:** Contains one or more object properties and requires at least two relationships.
 - c) **Composite object:** An object which is part of another object.
 - d) **Association object:** An object that establishes a relationship between two other objects and is contained in both of them.
 - e) **Aggregation object:** A group of objects that is treated as an entity, such as sets and arrays.
 - f) **Complex object:** An object such as an image, voice, etc, which is usually broken into new composite objects in a nested or hierarchical fashion.

Results of the Survey

The following is a survey of 15 current OODBMSs. Where applicable, the history and origins of the OODBMSs is given, the applications which are best suited for the OODBMS along with their operational environments are discussed, and an overview of the OODBMS is given and any special or unique features are described.

AVANCE

AVANCE, previously named OPAL (not to be confused with the OPAL language used in GemStone), was developed at the University of Stockholm and the Swedish Institute for Systems Development (SISU). The main goal of AVANCE was to develop a research prototype

that would support applications that require the history or versions of an object to be retained. Such applications include office information systems, VLSI, CAD, and CASE. Object versioning in AVANCE is used (1) to capture the history of an object; (2) to cope with the problem of changing type definitions and to eliminate the requirement to perform massive data reorganizations; and (3) for concurrency control, recovery, and reliability.

AVANCE supports persistent objects, nested transactions, object-version management, decentralization of both data and control, and a high-level language PAL, similar to Smalltalk. PAL supports typing, property inheritance, data abstraction, encapsulation, and both dynamic and static binding within a homogeneous environment, (Brett et al., 1989).

The architecture of AVANCE is designed to provide three levels of abstraction to implement. The first level, the object manager, offers functions for the identification and manipulation of packet (shared objects) and provides an interface for building interpreters. Virtual machine, the second level of abstraction, is the virtual machine interface provided by the pseudo-machine code interpreter. The third level of abstraction is the high-level language PAL. A compiler translates PAL into the pseudo-machine code of the virtual machine.

AVANCE uses version management for:

- 1) synchronizing access,
- 2) facilitating recovery, and
- 3) providing the means for more efficient concurrency.

	AVANCE	ENCORE/ObServer	EXODUS/EXTRA
Primary Uses	Office Information Systems, Fault Tolerant Systems	Cooperative CAD	—
Version Mgt	Yes	Yes	Yes
Recovery	Through version management	Log Based	Shadow Page Log
Transaction Mgt	Yes	Yes	Yes
Composite Objects		Yes	Yes
Multiple Inheritance	Yes	Yes (But name conflict resolution is not clear)	Yes
Concurrency/ Locking	2-Phase Comm. Protocol (Timestamps)	4 Lock Types	Yes Hierarchal 2-PL
Distributed Support	Fully Distributed	—	No
Dynamic Schema Evolution	Dynamic & Static Binding	Yes (most all features)	—
Multimedia Data	No	Yes	Yes
Language Interfaces	PAL (Similar to Smalltalk)	C	E (Extension to C++)
Relational/ O-O Based	O-O Based	O-O Based	O-O Based
Platforms	—	SUN3, SUN4, & other UNIX workstations DEC (VMS)	—
Operational Status	Research Prototype	University Development	University Prototype
Special Features	—	Change Notification, Object Manager	Change Notification, DB Generator

There are three ways in AVANCE that applications can use version control. The simplest way is to utilize the system-level version control. The second approach is to built an independent

version control for application. The third method is a hybrid one. AVANCE provide facilities to built a version control utilizing AVANCE version control while providing additional features. The capability to manage parallel version structure

ENCORE/ObServer

ENCORE/ObServer is a university research prototype developed at Brown University.

GemStone	IRIS	KIWI	O ₂	ObjectStore
Cooperative Environments	OIS, CAD/CAM, Knowledge-Based Systems	Knowledge-Based Systems	CAD/CAM, GIS, OIS, Business and Transaction Processing	Collaborative Environments
Yes	Yes	Yes	Limited	Yes
Shadow Pages	Via HP-SQL	—	Yes	Log Based
Yes	Yes	—	Yes	Yes
No	Constrained	Yes	Yes	Somewhat Constrained
No Planned for future	Yes	—	Yes	Yes
3 Lock Types Optimistic and Pessimistic	Yes	Yes	Yes Optimistic	2 Lock Types
Yes	—	—	Yes	No
Yes (limited)	Yes (limited)	Yes	Yes (limited)	Yes (limited)
Yes	No	—	Yes	Yes
C, C++, OPAL, Parc Place Smalltalk, Topaz command Line Interface	C, LISP, OSQL	OOPS+	C	C, C++
O-O Based	Relational	—	Relational	—
SUN3, SUN4, Apollo, Apple, DEC (VAX/VMS), Tektronix, IBM PCs,	HP 68000 Workstations	SUNVIEW	SUN Workstations, OS Version 4.0 or higher	SUN3, SUN4, Apollo, HP, DEC (Ultrix)
Commercial	Research Prototyp	Research Prototype	Commercial	Commercial
Change Notification	Private and Share Databases	Visualization Interface	Visualization Interface, Powerful query Language	Private and Shared Databases

is a very powerful feature for the design of different types of applications. AVANCE is designed to be a fully distributed object-oriented database, (Bjornestedt et al., 1989).

ture of ObServer is the model of long-duration transactions for cooperative users in CAD environments. Lock requests are accompanied by notification requests, so that the requestor of

ObServer is considered to be an "object manager" which is generally less functional than fully implemented OODBMSs.

Minimally, object managers provide a persistent object store, with concurrency control and recovery, and generally do not provide a query and programming language. Object managers may be useful in applications which desire very high performance without all the capabilities of a OODBMS (along with the associated overhead). In the case of ENCORE/ObServer, ObServer is used as the storage level of a full DBMS called ENCORE.

ObServer stores multimedia data and supports lock-based currency control, log-based recovery, and transactions. A noteworthy fea-

a lock, after acquiring a lock, may be notified if a subsequent requestor of the new conflicting lock is blocked from access to the desired object. ObServer also makes uncommitted changes of one transaction visible to other transactions, and even allows some changes to remain in the database for aborted transactions. For dead-locks, ObServer simply removes one of the transactions in the deadlock from the lock-wait queue, rather than aborting the transaction. This approach is motivated by the desire to avoid losing long-duration transactions.

The ENCORE OODBMS allows each object to be stored as a whole, not as multiple tuples spread across several relations. Each link object can be stored once, and the object need not be copied into the application's data structures. The ENCORE data model includes multiple inheritance, multi-valued attributes, and version management.

ObServer runs on SUN workstations under UNIX, while ENCORE runs on both SUN workstations and DEC machines under VMS.

EXODUS/EXTRA

The EXODUS database system generator was developed at the University of Wisconsin. The central idea is to offer a toolkit that simplifies the development of database management systems. EXODUS provides a set of kernel facilities for utilization across all applications, such as a versatile storage manager and a general-purpose manager for type-related dependency information. The EXODUS database system generator allows a database implementor to customize the data model itself, and to produce an object-oriented database management system, a relational database management system, or even a file management system providing only access methods. EXODUS provides the database implementor with a programming language, E (an extension of C++), the E compiler, a rule-based query optimizer generator, and tools for constructing query-language optimizers.

A DBMS generated by EXODUS consists of a parser and query optimizer constructed from the description of the target query language and operators, and the database engine itself. The output from the query optimizer is an E program, which is why the E compiler is needed.

The database engine performs buffer management, concurrency control, recovery, and version management.

An object-oriented database management system that was developed using the EXODUS system is EXTRA. EXTRA provides support for persistent objects, user-defined object types, multiple inheritance, user-defined methods for type encapsulation, and complex objects including lists, sets, and arrays. The EXTRA data model is a rich amalgamation and extension of the data models of O₂, GemStone, POSTGRES, GEM, and Orion (Kim, 1991).

EXODUS was one the first OODBMSs that provided capabilities allowing very large dynamic objects to run efficiently at a relatively low cost and at an extremely reasonable level of storage utilization. The underlying data structures (a B+ tree) and algorithms support large storage objects efficiently, both in storage utilization and in access and update performance. Object presidency and version management are space efficient. Once an object is modified to create a new version, only the modified pages of objects are copied (Kemper, 1994).

The storage system provides a procedural interface which includes methods for creating and destroying file objects and for scanning through the objects that they hold. The EXODUS storage system offers concurrency control and recovery services for storage objects. Initially, concurrency control is offered via hierarchical two-phase locking at the levels of file and storage objects. EXODUS utilizes shadowing and logging to perform recovery. Updated internal pages are shadowed up to root of the B+ tree to ensure the undo (or redo) process on large objects.

GemStone

GemStone was developed by Servio Logic Corporation of Alameda, CA, to satisfy the needs of commercial and MIS markets, as well as engineering markets. The initial approach taken was to examine the Smalltalk language and system and to identify a number of requirements to make Smalltalk a database system. The result is the language OPAL, which manipulates persistent data objects controlled by a disk-based storage manager (Brown, 1991).

GemStone offers application developers a data-

base subsystem with a Smalltalk-like object model rather than one of the more traditional record-oriented models (e.g., relational or hierarchical). This model allows applications to handle information such as documents, pictures, and sound which are not easily handled by more traditional database systems. GemStone's object model is exactly the same as Smalltalk-80; the three principal ideas of the GemStone object model are object, message, and class. The classes in GemStone are structured in a class hierarchy, with each subclass inheriting behavior and structure from its superclasses. For example, when a message is received by an object, the object consults with this class (and superclasses) to find the correct method for execution. A special feature for which GemStone was designed for is to increase the data-modeling power of a database component with the goal of speeding up the development time of applications such as office information systems, documentation of complex mechanical systems (e.g., automobiles, airframes), computer-aided design, and artificial intelligence knowledge bases.

There are two main components to the GemStone architecture. First, the Gem server process is where query evaluation occurs, and where object behavior specified in GemStone's DML is executed. Second, the Stone monitor allocates new object identifiers and provides persistent object management, concurrency control, authorization, transaction, and recovery services.

The Stone process typically resides on the server machine, while the Gem process may reside on the server or on a client workstation. Currently, a VAX, SUN3, or SUN4 workstation is used as the server, while IBM PCs and compatible, Apple Macintoshes, SUN3 and SUN4, and Tektronix workstations are supported as client machines.

In GemStone, both pessimistic and optimistic concurrency control schemes are implemented, either of which may be used depending on the situation. When contention is high, pessimistic concurrency control reduces wasted effort in aborting transactions at the expense of managing locks. When contention is low, optimistic concurrency control maximizes concurrency (Bretl et al., 1989).

GemStone also offers sharing of objects by giving each user a particular list of dictionaries known as a SymbolList. This list is private to each user, but the dictionaries it uses may be shared. Other features include fast associative access, uniform language, primary and secondary storage management, resilience to common failure modes, centralized server, and security.

Iris

The Iris database management system was developed as a research prototype at the Hewlett-Packard Laboratories in Palo Alto, CA, and is intended to meet the needs of database applications such as office information and knowledge-based systems, engineering test and measurement, and hardware and software design. The Iris data model is based on three major concepts: types, objects, and functions, and is very similar to the O₂ system. Iris supports inheritance and generic properties, constraints, complex and non-normalized data, user defined operations, version control, and extensible data types (Fishman et al., 1989).

One of the most interesting features of Iris is that the Iris database contains an object manager which is built on top of HP's relational database system, HP-SQL, which acts as the storage manager for the system. All Iris queries and functions are converted into an intermediate form of relational algebra. At this level, query optimization can take place based on conventional algebraic techniques.

The language of Iris is very powerful because it was defined with a syntax which recalls the syntax of SQL and has an expressive power comparable to that of SQL. Iris may be accessed via both interactive and programmatic interfaces. These interfaces are implemented using a library of C routines that define the Iris object manager interface. The first interactive interface, Object SQL (OSQL), is an extension to SQL. The second interactive interface, the graphical editor, allows a user to browse and update an Iris database. Iris currently supports embedded OSQL in common LISP and ANSI standard C.

The system supports transactions with save points and restores to save points, concurrency

control, logging and recovery, archiving, indexing, and buffer management (Fishman et al., 1989).

The Iris DBMS is implemented in C on HP-9000/350 UNIX workstations, which are MC68020-based computers.

KIWI

The KIWI object-oriented database management system was developed at the University of Antwerp, in Wilrijk, Belgium, as a prototype to support knowledge-base applications, and to provide an attractive user interface that can be developed in a very short period of time.

An object-oriented database programming language, OOPS+, was developed to be used with the KIWI system as an interface to the database. OOPS+ includes the notion of persistent objects, and provides a suitable tool for knowledge-based application programming, integrating concepts from semantic modelling, databases, logic, and object-oriented programming in a simple and orthogonal fashion (Laenens et al., 1989).

The user interface that was developed for use with the KIWI system is called Not Another User interface (NAU). The default interface, mostly written in OOPS+, associates a standard visualization with each object. It is then possible to customize the presentation of the objects that are relevant to the application. The customization can be done without any programming; an NAU design tool allows an interactive design of hierarchical window layouts and associates

them with the appropriate object or class. It is interesting to note that in NAU, an object is represented by a window, which itself is an object.

	ONTOS	ORION	OZ+
Primary Uses	CAD/CAM	CAD/CAM, AI, OIS, Multimedia Information	Office Information Systems
Version Mgt	Yes	Yes	Yes
Recovery	Yes	Combination of Logs & Shadow Pages	Periodical update & Write ahead
Transaction Mgt	Yes	Yes	
Composite Objects	No	Yes	Complex Objects
Multiple Inheritance	Yes	Yes	No
Concurrency/ Locking	4 Lock Types	5 Lock Types	Yes
Distributed Support	Yes	Yes (ORION-2)	Yes
Dynamic Evolution	Yes (Quite limited)	Yes (All features)	—
Multimedia Data Mgt	No	Yes	No
Language Interfaces	C++	LISP, C	OZ+
Relational/ O-O Based	O-O Based	O-O Based	Relational
Platforms	SUN3, SUN4, Apollo, OS/2, VAX (VMS)	Symbolics 3600 LIS machines, SUN3, Apollo, HP, SGI, DECstation	SUN3 (Clients) SUN4 (Server)
Operational Status	Commercial	Commercial (ORION-2)	University Prototype
Special Features	Object SQL	Change Notification, Private and Shared Databases	Language Based Support

The NAU currently runs on top of the SUNVIEW window package, and has also been integrated with the KIVIEW browser. Using the fact that windows are themselves objects, the KIWI team

is developing a "window-definition module" which can be used to design interface objects without programming.

as a research prototype and is now a commercially available product. The project was funded by INRIA, Siemens-Nixdorf, Bull, CRNS, and the University of Paris. The goal of the O₂ project was to develop a next-generation

OODBMS suitable for applications such as CAD/CAM, geographic and urban systems, editorial information systems, and office automation, as well as such traditional areas as business and transactions processing (Deux et al., 1991). O₂ is implemented in C and runs on SUN workstations under SUN OS 4.0.

POSTGRES	Starburst	Statice	VERSANT
CAD/CAM, Knowledge Based Systems	CAD/CAM, AI, Expert Systems, Knowledge Based Systems	—	Collaborative Engineering
Yes	No	Limited	Yes
No-Overwrite	Rollback	REDO Log	—
Yes	Yes	Yes	Yes
Yes	Complex Objects	Yes	Yes
Yes	Yes	Yes	Yes
Pessimistic & Optimistic	Rules and Rollbac	2-phase locking	2-Phase Locking 4 Lock Types
No	Yes	Yes	Yes
—	—	—	Yes (Limited)
Yes	Yes	Yes	No
C, C++, LISP	C, C++	Common LISP	C, C++
Relational	Relational	O-O Based	O-O Based
SUN3, SUN4, DECstation, Sequent Symmetry	IBM PCs, RISC 6000	Symbolics LISP Machines	SUN3, SUN4 (SUN OS Version 4.0 or higher)
University Prototyp	IBM Testbed	Commercial	Commercial
		Graphical browser Extensive development tools	Change Notification, Private and Shared Databases

The core of the O₂ system is the O₂Engine, which stores multimedia and structured objects, handles disk management, distribution, transaction management, concurrency, recovery, security, and data administration. The O₂ Engine supports complex objects with identity, classes, types, encapsulation, methods, multiple inheritance, overriding, and late binding of methods (Deus et al., 1991). It is composed of three main layers, a schema manager, an object manager, and the O₂ disk manager which is an extension of the Wisconsin Storage System (WiSS).

The O₂Engine can support two types of interfaces, the C and C++ language interfaces, and the O₂ environment. The O₂ environment includes (1) a query language, O₂Query, which is an extension to SQL which deals with complex values and objects; (2) a user-interface generator, O₂Look,

which supports the creation of simple but high quality graphical interfaces through the use of "ready made," easily customizable, object presentations on the screen; (3) an object-oriented

O₂

The O₂ object-oriented database system was developed at GIP Altair in Le Chasnay, France,

fourth-generation language (4GL), CO₂, which allows a user to perform programming, database manipulation, and user interface generation (through O₂Look); and (4) a graphic programming environment including a schema and database browser.

A notable drawback of O₂ is that it was initially developed to be embedded into existing programming languages. Thus, the O₂ model and its implementation language for methods do not have a smooth, integrated, orthogonal framework.

ObjectStore

ObjectStore, developed by Object Design of Burlington, MA, is a commercially available product, based on making C++ a database programming language. ObjectStore provides a tightly integrated language interface to the traditional DBMS features of persistent storage, transaction management, distributed data access, and associative queries. It was designed to provide a unified programmatic interface to both persistent and transient data, with object-access speed for persistent data equal to that of transient data (Lamb et al., 1991). The main target applications for ObjectStore are data-intensive programs that perform complex manipulations on large databases of objects such as CAD, CAE, CAP, CASE, and GIS.

ObjectStore has been implemented using C++ in a way such that any C++ data type, including built-in and user-defined data types, can be allocated transiently or persistently. Once allocated, manipulation of persistent data is no different than transient data, and incurs no performance overhead. The key to meeting these requirements is through ObjectStore's virtual memory mapping architecture, which supports uniform memory references to data regardless of whether the data is transient or persistent. When an ObjectStore application dereferences a pointer (ObjectStore uses 32-bit pointers directly as OID's) whose target has not been retrieved into the client, the hardware detects an access violation, and the operating system reflects this back to ObjectStore as a memory fault.

ObjectStore currently runs on Sun3 and SPARC under SunOS, IBM RS/6000 under AIX, DEC DS3100 under Ultrix, HP 300/400/700 under P/UX, DEC under VMS, and Microsoft Windows 3.0.

Ontos

Ontos, developed by Ontologic of Billerica, MA, is a commercially available product based on making C++ a database programming language.

Ontos replaces an earlier product from Ontologic, VBase, which used the object-oriented model with proprietary languages COP and TDL. The non-standard languages and performance problems of VBase led Ontologic to introduce Ontos. Ontos has been implemented using C++ and runs under UNIX on SUN3 and SUN4, Apollo, and DEC workstations, and is also available for the OS/2 operating system.

Ontos relies heavily on the power and portability of C++. As is true with most C++ based systems, transactions are defined statically at compile time; it does not provide a flexible runtime environment. C++ provides Ontos with other object-oriented features such as true encapsulation by data hiding (in the form of private and protected variables), and multiple inheritance. For example, Ontos allows items that are stored in the database to be accessed and manipulated within a C++ application program. In other words, database objects can be "activated" (transferring an object's state from the secondary-storage base into the virtual memory of the C++ application) and objects can be "deactivated" (moved back into database).

Ontos serializes all concurrent transactions, and provides both optimistic and pessimistic concurrency control schemes. In a pessimistic situation, access conflicts are checked at the time a transaction is initiated; this reduces concurrency, but ensures transaction commits when it eventually finishes. Ontos allows checkpointing of transactions before commit. Checkpointing commits all the objects in the transaction buffer pool immediately to the database without releasing the locks.

The Ontos object database is based on the client/server architecture for data interaction in which the server side runs the data store, and the client side offers the interface to user processes and oversees the mapping of data to the application processes' virtual memory space. The client and server can communicate over a local area network.

One special feature of Ontos is that it facilitates two extremely different types of inter-object

references: direct C++ references and abstract references. In the former, the maintenance of the reference (taking precautions that the referenced object is brought into the main memory) is delegated to the programmer. In the latter, abstract references allows you to maintain the reference when the abstractly referenced object is moved within the main memory whereas in direct referencing, this is the responsibility of the user. Another feature of Ontos is that the user may select between convenience and safety on the one hand and performance on the other. The user decides the appropriate tradeoff between safety and performance because Ontos does not stop the user from "breaking the rules," for example, substituting safe abstract references by direct references.

Ontos is one of the first "persistent" OODBMSs. The versioning mechanism of Ontos allows a single object to exist in one or more number of versions. The versioning mechanism collects all objects of the same version in a configuration object. To create a new configuration (e. g., an application) from existing configurations links between related configuration are derived and object need not to be copied.

Although Ontos has a large installation base and is one of the most widely used C++ based OODBMSs, the earlier versions of Ontos had some major drawbacks. It provided little or no support for multimedia, composite objects, dynamic schema evolution, change notification, or security. These limitations have been recognized and most are being addressed in the next release of the product (Kemper, 1994).

Orion (Itasca)

Orion was developed in the Advanced Computer Technology (ACT) program at Microelectronics and Computer Technology Corporation (MCC) in Austin, TX. The original intent of Orion was to provide the data management support of Proteus, an expert system shell project developed within the same program. Orion has evolved through a series of prototypes; and the most recent version, Orion-2, a fully distributed one, is being offered as a commercially available product from Itasca Systems, Inc., of Minneapolis, MN (Kemper, 1994).

Functions supported by Orion include object identifiers (OIDs), multiple inheritance, composite objects, versions and change notification, queries, transaction management, distributed databases, dynamic schema evolution, and multiple-media data management. Orion has been implemented in common LISP and runs on either a Symbolics 3600 LISP machine, or on a network of SUN3 workstations running under the UNIX operating system. Because of the features offered by Orion, it is being marketed toward data-intensive applications such as CAD/CAM, artificial intelligence, and office information systems Kim, (1989).

One major architectural innovation in Orion-2 is the coexistence of a shared database and a number of private databases. The shared database is accessible to all authorized users of the system, while each private database is accessible to only the user who owns it. Users may check out selected objects from the shared database, which copies the objects and inserts them into the private database. When the user checks the objects back into the shared database, the objects are moved, not copied, to retain the same OID as the checked-out version (if the object is not versionable or modified). The shared database is viewed as extension of the private. Therefore, queries in Orion could be distinguished into three types. Private queries are issued for private databases, shared queries are those used for shared database, and global queries are issued for both type of databases. Some of the significant features of Itasca are (1) ability to reconfiguration dynamically multiple client and server nodes in a network with no central server; (2) support for multiple inheritance; several conflict resolution strategies are provided to resolve conflicts in multiple inheritance; (3) support for both classical and long-duration transaction; (4) providing recovery schema for CPU failure; and (5) providing a comprehensive version management that support applications with compound documents and multiple versions in integrated systems (Kemper, 1994).

OZ+

OZ+ was developed at the University of Toronto as an OODBMS prototype that would meet the needs of office information system applications.

The direction of OZ+ was greatly influenced by another project being developed at the University of Toronto, the Office Task Manager (OTM); OZ+ was implemented as the underlying DBMS of the OTM project.

OZ+ was written in C and Turing Plus under version 4.2bsd of the UNIX operating system. It runs in a client/server environment with SUN3/50 clients and a SUN3/280 file server. OZ+ utilizes the EMPRESS relational database management system as the underlying disk storage and access mechanism. Algorithms have been developed to encode an OZ+ object into a set of relational tuples, as well as for the inverse. These algorithms are fast and do not contribute significantly to the overhead of swapping objects back and forth between memory and disk (Weiser et al., 1989). The amount of time spent locating a particular object instance in a set of objects of the same type is the same as the time spent to find a particular tuple in a relation.

One interesting feature of OZ+ is that it supports a notion of self-triggering rules. An object which contains a self-triggering rule will invoke itself and perform its actions when its conditions all become true. The object containing a self-triggering rule appears to act spontaneously; that is, it will perform its actions without having received a message telling it to do so.

In the event of a crash, recovery begins with a message from the recovery manager indicating that the system has just "restarted." Performing periodical updates and the existence of "write ahead" minimizes the task of recovery and guarantees the consistency of objects. Access to all objects is to be cleared through an event manager. If any process fails, the event manager forces the related event also fail. Should the event manager fail, all partially completed events are discarded.

POSTGRES/Miro

POSTGRES, considered by some to be the most powerful extended relational database system currently implemented, was developed at the University of California at Berkeley. POSTGRES was named as a follow-on to the Berkeley work on INGRES. The POSTGRES system provides objects, OIDs, compound objects, multiple inheritance, versions, historical data, procedures, and a powerful extended re-

lational query language, POSTQUEL, named after INGRES QUEL.

The fundamental goal of POSTGRES is to provide support for applications which require the services of object and knowledge management. Object management, which deals with efficiently storing and manipulating non-traditional data types such as bitmaps, icons, text, and polygons, is a typical requirement of CAD and other engineering applications (Stonebraker et al., 1991). Knowledge management deals with the ability to store and enforce a collection of rules which describe integrity constraints about the application, as well as allowing data that is not directly stored in the database to be derived.

POSTGRES uses the approach of extending to the relational model through adding new data types, operators, and new access methods. This feature preserves the integration of POSTGRES with other existing relational databases and provides a smooth flow between engineering and business applications. The design of POSTGRES was directed toward five main goals: (1) to provide better support for complex objects; (2) to provide user extensibility for data types, operators, and access methods; (3) to provide facilities for active databases (i.e., alters and triggers) and inferencing including forward and backward-chaining; (4) to simplify the DBMS code for crash recovery; and (5) to make as few changes as possible for the relational data model. POSTGRES could be accurately described as an object-oriented system because it includes unique identity for objects, abstract data types, classes, methods, and inheritance for both data and functions. One of the important aspects of POSTGRES is its support for historical data. POSTGRES is one of the few systems designed to support the temporal dimension of storage. Another aspect of POSTGRES design is the incorporation of new technology into its architecture. The use of optical disks as archival storage to store committed history data adds security and simplicity factors in case of a crash recovery situation.

One interesting feature of POSTGRES is the implementation of a "no-overwrite" storage manager. When updates occur, the old historical data is archived onto optical disks and serves the purpose normally performed by a write-ahead log. This provides two desirable

features: instantaneous crash recovery and time travel. Aborting a transaction can be instantaneous because no "UNDO" of transaction from the log is required; the previous records are readily available in the database (Stonebraker, 1991).

The POSTGRES system, which consists of 180,000 lines of C code, was designed to be called from many different programming languages, and currently has interfaces in common LISP and C++. It runs on SUN3, SUN4, DECstation, and the Sequent Symmetry machines and can be obtained free of charge over the Internet or on a tape for a modest reproduction fee (Stonebaker et al., 1991). Miro is a commercial version of POSTGRES.

Starburst

Starburst is an extended relational database system developed at the IBM Almaden Research Center. The goal of Starburst was to build from scratch an extensible DBMS prototype that would (1) facilitate enhancing DBMSs with the functionality to support new applications, especially those not currently served well by commercially available DBMSs; and (2) provide a flexible testbed for ongoing research and experiments in new DBMS technology (Lohman et al., 1991).

For portability, Starburst was written in C. Development of Starburst was started on the IBM PC/AT platform, and the portability was proven by migrating to the AIX operating system (IBM's version of Unix V), and later to the new RISC System/6000 computers.

One of the biggest limitations of traditional relational DBMSs is their fixed set of data types and functions on those data types. Starburst allow user-defined data types and functions, as well as the concepts of abstraction and encapsulation hierarchies of user-defined types, and inheritance of functions on those types.

Another limitation of traditional relational DBMSs is their inability to store large (complex) objects such as images, voice, or video. Extensions to Starburst allows for two alternatives for storing complex objects. First, the entire object can be stored in a "long field" (up to 1.5 gigabytes) in which the representation of the object is entirely under user control. This greatly limits

the ability to perform queries against "long fields." The second alternative is to store the object's atomic components (if possible) as rows in a table using Starburst's extended Normal Form (XNF). XNF objects may be combined, projected, and restricted.

Stalice

Stalice is implemented in common LISP and runs on the Symbolics LISP machines. It supports a client/server architecture and can support multiple users, although the system is not a distributed system. It is designed to integrate seamlessly a distributed LISP programming environment with a database system (Kim, 1991). The Stalice data model is similar to Orion.

Stalice provides a fairly rich set of database features, including transaction management, concurrency control, recovery, a query language, query optimization, indexing, and multimedia data management. It uses page-level locking for concurrency control, and keeps a REDO log for recovery. Its transaction model is the conventional short-duration transaction model. It includes queries on a single and multiple target classes (Kim, 1991).

The Stalice type system maps directly into the common Lisp-type system, so the well-known common Lisp operators such as type, typecase, and check-type can be used in the normal manner for entity handles. Programs using Stalice can fix limits on entity classes, just like any other class. Therefore, Stalice fits freely into the object-oriented programming system that is already provided by the Lisp language.

The Stalice query language is a natural extension of the LISP syntax. It also supports the storage and retrieval of multimedia data (Kim, 1991).

Stalice makes an explicit distinction between read-only attributes and read-write attributes. It supports inverse functions on attributes. For example, if the user defines a class "Vehicle" with an attribute "Weight," the user may also define an inverse function on Weight. The system can then return a set of vehicles with a given weight. It allows explicit deletion of a persistent object; however, once deleted, it removes all references to that object, thereby preventing dangling references (Kim, 1991).

VERSANT

VERSANT is a commercially available OODBMS developed by Versant Object Technology (formerly Object Sciences) of Menlo Park, CA. VERSANT has been implemented in C/C++ 2.0, and currently runs only on the SUN3 and SUN4 platforms under SUN OS Version 4.0 or higher. VERSANT has programming interfaces to both C and C++.

VERSANT provides a reasonably powerful transaction management that supports two types of transactions: short and long transactions. Long transactions are persistent, can last for any amount of time, and can span multiple short transactions and processes. Long transactions provide a way for locks to be shared among short transactions. Short transactions are atomic in nature; a short commit or checkpoint follows a two-phase commit protocol. Implicit locks obtained in short transactions are not persistent because lock information is lost if the system crashes. Changes made during a short transaction can be made permanent by issuing a "checkpoint," which will retain all locks and objects in cache memory (Ahmed et al., 1992).

VERSANT provides reasonably flexible concurrency control. Several types of locks are provided: read, write, snapshot, and monitor locks. The snapshot lock creates a copy of the object without regard to the lock status of the object. The monitor allows a specification of events which will cause the user to be notified if one such event occurs. With multiple types of locks and various ways of using them, serializability can be bypassed, if needed, to increase concurrency. Database consistency must be maintained by the application in this case.

Conclusion

With approximately 30 OODBMSs in various stages of development and refinement around the world, core O-O concepts are shared by almost all systems. That is, the data models include the notions of object identity, instances, classes, class hierarchy and inheritance, and message passing. Almost all systems support nested attributes and set-valued attributes; however, some systems support a more general notion of multi-valued attributes than others. All systems support persistent objects, while there

are different notions regarding explicit deletion of persistent objects. Most systems currently support a client/server architecture to support multi-user workstations environments. Further, most systems support concurrency control, recovery, and transaction management. Some systems do so through the features of underlying relational storage systems, while others provide these features directly. There are also strong indications that in the near future most systems will include multiple inheritance, versioning, queries, and multimedia data management. Some systems already support some or all of these features.

The fifth-generation database technology currently being developed will help deal with many real-world conditions, such as computer-aided manufacturing (CAD/CAE/CASE/CAM), compound document composition, knowledge-based systems, multimedia information systems, and advanced human-interface systems.

In general, because of object-oriented model complexity and such additional features, such as versioning, we cannot expect OODBMSs to be highly efficient; however, the first relational DBMSs had much the same problem. With more research and development, we can expect OODBMSs which are quite efficient in the near future.

References

- Ahmed, Shamim et al. (1992). "Object-Oriented Database Management Systems for Engineering: A Comparison," *Journal of Object-Oriented Programming*, June 1, Vol. 5, No. 3.
- Atwood, T. (1994). "ODMG-93: the Object DBMS Standard," *OBJECT*, January.
- Bertino, Elisa and Martino, Lorenzo (1991). "Object-Oriented Database Management Systems: Concepts and Issues," *Computer*, April.
- Bretl, Robert et al. (1989). "The Gemstone Data Management System," In *Object-Oriented Concepts, Databases, and Applications*, Kim and Lochovsky, (Eds.), Addison-Wesley, Reading, MA.
- Brown, A. (1991). *Object-Oriented Databases: Applications in Software Engineering*, McGraw-Hill Book Company.
- Bjornerstedt, Anders and Hulten, Christer (1989). "Version Control in an Object-Oriented Architecture," In *Object-Oriented Concepts*,

Databases, and Applications, Kim and Lochovsky, (Eds.), Addison-Wesley, Reading, MA.

Cattell, R. (1991). *Object Data Management: Object-Oriented and Extended Relational Database Systems*, Addison-Wesley Publishing Company.

Deux, O. et al. (1991), "The O₂ System," *Communications of the ACM*, October, Vol. 34, No. 10.

Fishman, D. H. et al. (1989). "Overview of the IRIS DBMS," In *Object-Oriented Concepts, Databases, and Applications*, Kim and Lochovsky, (Eds.), Addison-Wesley, Reading, MA.

Kemper, A. and Morekotte, G. (1994). *Object-Oriented Database Management*, Prentice Hall, Englewood Cliff, New Jersey.

Kim, W., et al. (1991). "A Distributed Object-Oriented Database System Supporting Shared and Private Databases," *ACM Transactions on Information Systems*, January, Vol. 9, No. 1.

Kim, W. (1990). *Introduction to Object-Oriented Databases*, The MIT Press, Cambridge, MA.

Kim, W. et al. (1989). "Features of the ORION Object-Oriented Database System," In *Object-Oriented Concepts, Databases, and Applications*, Kim and Lochovsky, eds., Addison-Wesley, Reading, MA.

Laenens, E., and Verveir, D. (1989). "Browsing a la carte in Object-Oriented Databases," *The Computer Journal*, Vol. 32, No. 4.

Lamb, Charles et al. (1991). "The OBJECT-STORE Database System," *Communications of the ACM*, October, Vol. 34, No. 10.

Lohman, Guy M. et al. (1991). "Extensions to STARBURST: Objects, Types, Functions, and Rules," *Communications of the ACM*, October, Vol. 34, No. 10.

Ozsu, M. Tamer, and Valduriez, Patrick (1991). "Distributed Database Systems: Where Are We Now?" *Computer*, August.

Skarra, Andrea H., and Zdonik, Stanley B., (1989). "Concurrency Control and Object-Oriented Databases," In *Object-Oriented Concepts, Databases, and Applications*, Kim and Lochovsky, eds., Addison-Wesley, Reading, MA.

Stonebraker, Michael, and Kernnitz, Greg (1991). "The POSTGRES Next-Generation Database Management System," *Communication of the ACM*, October, Vol. 34, No. 10.

Vorwerk, R. (1994). "Toward a True ODBMS," *OBJECT*, January.

Weiser, Stephen P., and Lochovsky, Frederick H. (1989). "OZ+: An Object-Oriented Database System," In *Object-Oriented Concepts,*

Databases, and Applications, Kim and Lochovsky, (Eds.), Addison-Wesley, Reading, MA.

Addresses of Producing Organizations

Avance:

Department of Computer and System Sciences, University of Stockholm, Stockholm, Sweden.

Encore:

Department of Computer Science, Brown University, Providence, RI, USA.

Exodus:

Computer Science Department, University of Wisconsin, Madison, WI, USA.

GemStone:

Servio Logic Corporation, 1420 Harbor Bay Parkway, Suite 100, Almaden, CA 94501, USA.

Itasca:

Itasca System Inc., 2850 Metro Drive, Suite 300, Minneapolis, MN 55425, USA.

IRIS:

Hp Laboratories, 1501 Page Mill Rd., Palo Alto, CA 94303, USA.

KIWI:

University of Antwerp, Wilrijk, Belgium.

O₂:

O₂ Technology, 7, Rue du Parc de Calgary, 78000 Versailles, Paris, France.

ObjectStore:

Object-Design Inc., 1 New England Executive Park, Burlington, MA 01803, USA.

Ontos

Ontologic Inc., 3 Burlington Woods, Burlington, MA 01801, USA.

Orion:

Microelectronic and Computer Technology Corporation, Austin, TX, USA.

OZ+:

Computer Systems Research Institute, University of Toronto, Toronto, Canada.

POSTGRES:

Computer Science Department at University of California at Berkeley, Berkeley, CA, USA.

Starburst:

IBM Almaden Research Center, 650 Harry Rd., San Jose CA, 95120 USA.

Statice:

Symbolic Inc., 1 New England Executive Park, Burlington, MA 01803, USA.

Versant:

Versant Object Technology, 4700 Bohannon Drive, Menlo Park, California.